JAVA

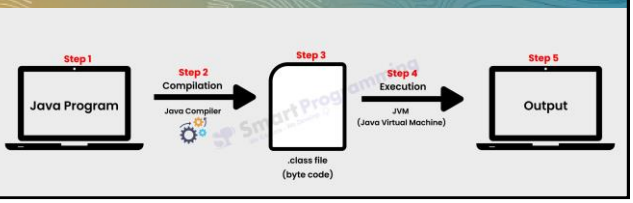
DESENVOLVIMENTO DE APLICAÇÕES

Prof. Leonardo Coelho | 2026

TEMA 01







1

1. A Estrutura Básica de um Programa Java

Todo programa em Java precisa de uma **estrutura mínima** para ser executado.
O ponto de partida padrão é a definição de uma classe e do método principal (**main**).

```
public class Main {
    public static void main(String[] args) {
      System.out.println("Hello, World!");
    }
}
```

- **public:** Modificador de acesso que torna a classe visível para qualquer outra parte do programa. Em Java, se uma classe é pública, o arquivo deve ter exatamente o mesmo nome dela (ex: **Main.java**).
- **class:** Palavra-chave usada para declarar uma classe, que funciona como o "molde" ou a estrutura principal do seu código.
- **Main:** O nome dado à classe. Por convenção, inicia-se com letra maiúscula.

2

DESENVOLVIMENTO DE APLICAÇÕES

1. A Estrutura Básica de um Programa Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

- **Chaves { }:** Delimitam o início e o fim de um bloco de código. Tudo o que pertence à classe ou ao método deve estar entre elas.
- **public static void main(String[] args):** É o ponto de entrada (motor) do programa. O Java procura por essa linha exata para saber por onde começar a execução.
- **static:** Permite que o método seja chamado sem criar um objeto da classe.
- **void:** Indica que o método realiza uma ação mas não retorna nenhum valor.
- **main:** É o nome do método. Significa "principal".
- **String[] args:** Parâmetro que permite receber argumentos via linha de comando.

3

DESENVOLVIMENTO DE APLICAÇÕES

1. A Estrutura Básica de um Programa Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

- **System.out.println(...):** Comando responsável por exibir uma mensagem no terminal (saída padrão) e pular para a próxima linha.
- **Ponto e Vírgula (;):** Indica o fim de uma instrução em Java. Sua omissão causa erro de compilação.

4

DESENVOLVIMENTO DE APLICAÇÕES

1. A Estrutura Básica de um Programa Java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

RESUMO VISUAL:

Se fôssemos traduzir essa linha para o português estruturado, seria algo assim:

- Crie uma **estrutura pública** (acessível a todos) do **tipo Classe**, e o nome dela será **Main**.
- Logo após essa linha, você costuma ver uma abertura de **chaves {.**
- Tudo o que estiver **dentro dessas chaves { ... }** pertence à **classe Main**, incluindo as **funções (métodos) e variáveis** do seu programa.

5

DESENVOLVIMENTO DE APLICAÇÕES

2. Variáveis e Tipos de Dados

Tipo de Dado	Descrição	Exemplo de Sintaxe
String	Armazena sequências de caracteres (textos). Deve usar aspas duplas.	String nome = "Leonardo";
int	Armazena números inteiros (positivos ou negativos, sem casas decimais).	int idade = 30;
double	Armazena números com ponto flutuante (casas decimais). Usa-se ponto (.).	double altura = 1.75;
boolean	Armazena valores lógicos: verdadeiro (true) ou falso (false).	boolean ativo = true;

6

DESENVOLVIMENTO DE APLICAÇÕES

2. Variáveis e Tipos de Dados

Regras para nomear variáveis:

- ✅ Deve começar com letra, `_` ou `$`
- ✅ Pode conter números após o primeiro caractere
- ✅ Não pode conter espaços
- ✅ Não pode utilizar palavras reservadas da linguagem

Evite nomes genéricos como:

- ❌ `x`
- ❌ `a1`
- ❌ `teste`

Boas práticas

Utilize nomes claros e significativos para facilitar a leitura e manutenção do código:

- ✔ `quantidadeProdutos`
- ✔ `valorTotal`
- ✔ `nomeCliente`

7

DESENVOLVIMENTO DE APLICAÇÕES

2. Variáveis e Tipos de Dados

Java

```java
public class Main {
    public static void main(String[] args) {
        // Criando as variáveis (as caixas)
        String nome = "Leonardo";
        int idade = 30;
        double altura = 1.75;

        // Usando as variáveis para exibir uma mensagem personalizada
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade + " anos");
        System.out.println("Altura: " + altura + "m");
    }
}
```

8

DESENVOLVIMENTO DE APLICAÇÕES

3. Estruturas Condicionais (Tomada de Decisão)

As estruturas condicionais permitem que o programa execute diferentes blocos de código baseando-se em **condições lógicas avaliadas como verdadeiras ou falsas**.

O comando if / else

```
int idade = 20;

if (idade >= 18) {
    System.out.println("Você é maior de idade.");
} else {
    System.out.println("Você é menor de idade.");
}
```

- **if:** Avalia a condição entre parênteses.
  Se for verdadeira (true), o bloco de código imediatamente abaixo é executado.
- **else:** Funciona como um plano alternativo.
  Caso a condição do if seja falsa (false), o bloco do else é o que será executado.

9

DESENVOLVIMENTO DE APLICAÇÕES

4. Estruturas de Repetição (Loops)

Loops são utilizados para repetir um bloco de instruções várias vezes de forma automatizada, evitando a repetição desnecessária de linhas de código.

O laço for

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Contagem: " + i);
}
```

O for é dividido em três partes essenciais separadas por ponto e vírgula:

1. **Inicialização (int i = 1;)**: Cria a variável contadora e define o seu valor inicial.
2. **Condição (i <= 5;)**: O loop continuará executando enquanto esta expressão for verdadeira.
3. **Incremento (i++)**: Adiciona +1 ao valor do contador ao final de cada repetição.

10

DESENVOLVIMENTO DE APLICAÇÕES

5. Operadores Matemáticos

Operador	Operação	O que faz?	Exemplo em Java	Resultado
+	Soma	Soma dois valores (ou junta textos).	int x = 5 + 3;	8
-	Subtração	Subtrai o segundo valor do primeiro.	int x = 10 - 4;	6
*	Multiplicação	Multiplica dois valores.	int x = 4 * 3;	12
/	Divisão	Divide o primeiro valor pelo segundo.	double x = 10.0 / 4.0;	2.5
%	Resto da Divisão (Módulo)	Devolve o que sobra de uma divisão inteira.	int x = 5 % 2;	1 *(pois 5 ÷ 2 é 2 e sobra 1)*

11

DESENVOLVIMENTO DE APLICAÇÕES

5. Operadores Matemáticos (Regras de Ouro)

1. Cuidado com a divisão de números inteiros!
Se você tentar dividir dois números inteiros, o Java vai truncar o resultado e esquecer as casas decimais.

- `int resultado = 5 / 2;` → O resultado será `2` (ele joga fora o `.5`).
- Para obter o resultado quebrado correto (`2.5`), pelo menos um dos números precisa ter ponto flutuante: `double resultado = 5.0 / 2;`

2. Ordem de Precedência (Quem vai primeiro?)
O Java segue a regra padrão da matemática: ele faz primeiro as multiplicações (*), divisões (/) e restos (%), e só depois faz as somas (+) e subtrações (-).

- `double media = 7.0 + 6.0 + 8.0 / 3;` → **Errado!** O Java vai dividir o `8.0 / 3` primeiro e depois somar o resto.
- `double media = (7.0 + 6.0 + 8.0) / 3;` → **Correto!** Os parênteses forçam o Java a somar tudo primeiro para só depois dividir.

12

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR

Calculador de Média (Sistema de Notas)

- Crie uma variável para armazenar o nome de um aluno.
- Declare três variáveis do tipo double para três notas diferentes.
- Calcule a média aritmética: (nota1 + nota2 + nota3) / 3.
- Use **if/else** para exibir se o aluno foi:
  aprovado (média maior ou igual a 7.0) ou se ficou de **recuperação**.



13

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Possível Solução

Calculador de Média (Sistema de Notas)

Notas.java

```
1   public class Notas {
        Run | Debug
2       public static void main(String[] args) {
3
4           String nome = "Leonardo Coelho";
5           double nota01 = 9;
6           double nota02 = 5;
7           double nota03 = 10;
8           double soma = (nota01 + nota02 + nota03)/3;
9
10          if(soma >= 7.0) {
11              System.out.println(nome + ": Você está APROVADO");
12          }
13          else {
14              System.out.println(nome + ": Você está de RECUPERAÇÃO!!!");
15          }
16      }
```



14

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR

Mini Tabuada

- Crie uma variável inteira chamada numero com o valor da tabuada desejada (ex: 7).
- Utilize um **laço for** que varie de 1 a 10.
- Multiplique o número pelo contador atual e exiba o resultado formatado no console **(exemplo: 7 x 1 = 7).**

```
7 X 1 = 7
7 X 2 = 14
7 X 3 = 21
7 X 4 = 28
7 X 5 = 35
7 X 6 = 42
7 X 7 = 49
7 X 8 = 56
7 X 9 = 63
7 X 10 = 70
```



15

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Possível Solução

Mini Tabuada

```
7 X 1 = 7
7 X 2 = 14
7 X 3 = 21
7 X 4 = 28
7 X 5 = 35
7 X 6 = 42
7 X 7 = 49
7 X 8 = 56
7 X 9 = 63
7 X 10 = 70
```

Calcular.java

```
1   public class Calcular {
        Run | Debug
2       public static void main(String[] args) {
3
4       int num = 7;
5
6       for (int soma = 1; soma <= 10 ; soma++ ) {
7           int res = soma * num;
8           System.out.println(num + " X " + soma + " = " + res);
9       }
10
11      }
12  }
```



16

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR

Salário Semanal

- Escreva um programa para calcular e mostrar o salário semanal de uma pessoa, determinado pelas condições que seguem.
- Se o número de horas trabalhadas for inferior a 40, a pessoa recebe R$15,00 por hora,
- senão a pessoa recebe R$600,00 mais R$21,00 para cada hora trabalhada acima de 40 horas.



17

DESENVOLVIMENTO DE APLICAÇÕES

PARA PRATICAR | Possível Solução

Salário Semanal

```java
public class Main {
    public static void main(String[] args) {
        double horas = 30;

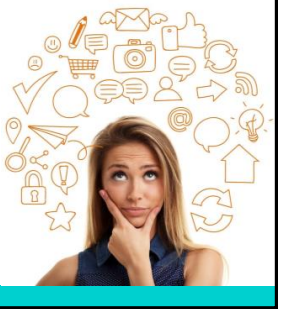
        double salario = 0;

        // Condicional limpo e direto
        if (horas < 40) {
            salario = horas * 15;
        } else {
            salario = 600 + ((horas - 40) * 21);
        }

        System.out.printf("Salário semanal: R$ %.2f%n", salario);

    }
}
```

```
Salário semanal: R$ 450.00
```



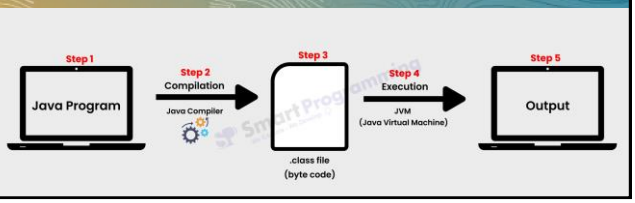
18

JAVA

DESENVOLVIMENTO DE APLICAÇÕES







19